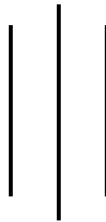




Tribhuvan University

Faculty of Humanities and Social Sciences



Cloud Computing Lab Report

Submitted To

Saroj Ojha

*In partial fulfilment of the requirements for the degree of bachelor's in
computer application.*

Submitted By

Homanath Adhikari Khatri

[6-2-1152-28-2022]

2026

Under the Supervision of

Saroj Ojha

Lecturer

Table of Content.

Lab 1: Introduction to Cloud Computing	1
Lab 2: Implementation of Virtualization using Oracle VM VirtualBox and a Guest Operating System	4
Lab 3: Running C and Java Programs in a Type 2 Virtual Machine.....	7
Lab 4: Installation and Configuration of Hadoop.....	9
Lab 5: Simple Word Count using Hadoop MapReduce	12
Lab 7: Case Study – Google App Engine.....	19
Lab 8: Case Study – Disaster Recovery Center (DRC), Hetauda	22

Lab 1: Introduction to Cloud Computing

Objectives

- To understand the basic concept and purpose of cloud computing.

Theory

Cloud computing is a model for delivering computing resources and services over a network, commonly the Internet. Instead of purchasing and maintaining all computing infrastructure locally, organization can obtain resources such as compute, storage, databases, networking, and software as services. The supplied report describes the provider as owning and managing the underlying hardware while the customer can request resources according to need.

Essential Characteristics

Characteristic	Explanation
On-demand self-service	Users can provision computing capabilities when required without manual interaction with the service provider.
Broad network access	Services are accessible through networks using devices such as laptops, phones, and other clients.
Resource pooling	Provider resources are shared among multiple customers using a pooled infrastructure.
Rapid elasticity	Resources can be increased or decreased according to demand.
Measured service	Resource consumption is monitored and can be used for usage-based accounting or billing.

Cloud Service Models

Model	Customer receives	Example
SaaS	A complete application delivered through the cloud.	Gmail
PaaS	A managed application platform for building and running applications.	Google App Engine
IaaS	Basic infrastructure such as virtual machines, storage and networking.	Amazon EC2

Cloud Service Models

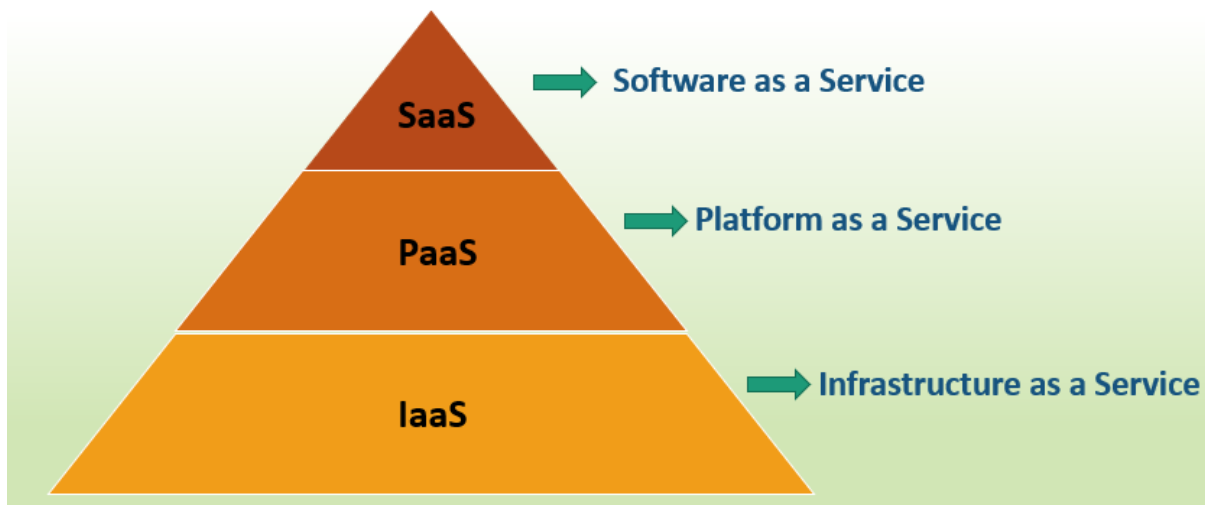


Figure 1: Cloud service models

In SaaS, the customer primarily uses the application. In PaaS, the customer focuses on application development and deployment while the provider manages much of the underlying platform. In IaaS, the customer has greater control over the virtual infrastructure and is responsible for more of the software stack. The source report uses these three models in the same order.

Cloud Deployment Models

Model	Description
Public cloud	Infrastructure is offered for use by the general public and operated by a cloud provider.
Private cloud	Cloud infrastructure is dedicated to a single organization and may be operated internally or by a third party.
Community cloud	Infrastructure is shared by organizations with common requirements or policies.
Hybrid cloud	Two or more distinct cloud environments are connected so that workloads or data can be coordinated across them.

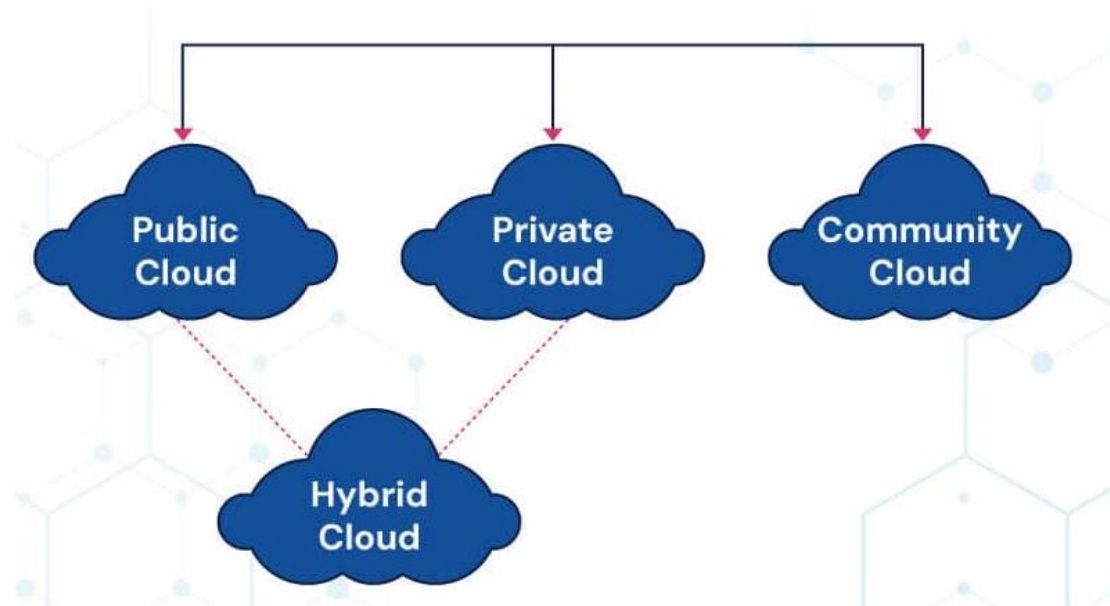


Figure 2: Cloud deployment types

The choice of cloud model depends on the required level of control, management responsibility, scalability, and operational effort. A user of SaaS normally manages application-level data and account settings, while an IaaS customer manages more of the operating-system and application stack.

Conclusion

In this lab, the fundamental concepts of cloud computing were studied, including essential characteristics, service models, and deployment models. The activity established the foundation required for understanding the virtualization, distributed-computing, SaaS, PaaS, and disaster-recovery experiments that follow.

Lab 2: Implementation of Virtualization using Oracle VM VirtualBox and a Guest Operating System

Objectives

- To understand the concept of virtualization and virtual machines.
- To create a virtual machine using Oracle VM VirtualBox.

Theory

Virtualization creates software-based representations of computing resources so that multiple virtual machines can operate on one physical host. The supplied report explains that a virtual machine runs above a hypervisor and that multiple guest operating systems can share the physical computer.

Terminology improvement is important here: creating a VM in VirtualBox is not, by itself, proof that the experiment implemented pure paravirtualization. VirtualBox is a hosted (Type 2) hypervisor and provides a paravirtualization interface for supported guest systems. Therefore, this report describes the activity more precisely as virtualization using VirtualBox with a paravirtualization interface, rather than claiming that the entire VM is exclusively paravirtualized.

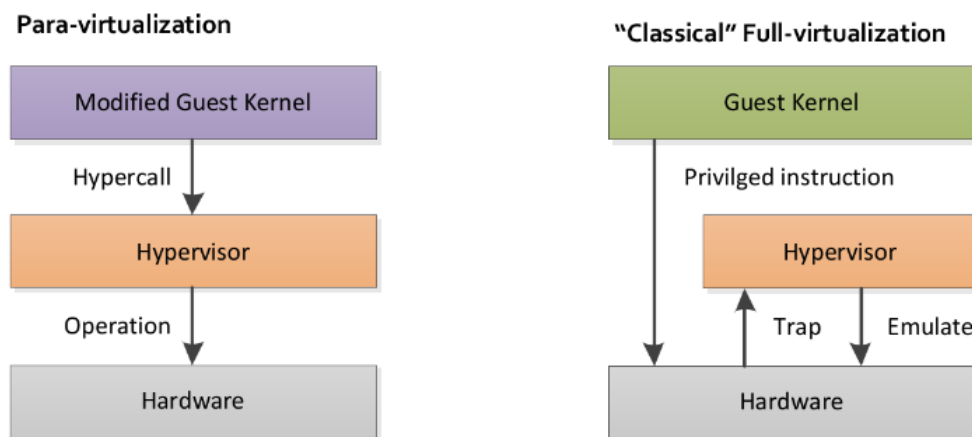


Figure 3: Conceptual comparison of paravirtualization and full virtualization

Steps to Create a Virtual Machine

1. Download and install Oracle VM VirtualBox on the host operating system.
2. Download the Ubuntu 22.04 LTS desktop ISO image.
3. Open VirtualBox Manager and select New.
4. Enter a descriptive VM name and select the Ubuntu ISO image.

5. Allocate appropriate RAM and CPU resources without exhausting the host machine.
6. Create and configure the virtual hard disk.
7. Start the virtual machine and complete the Ubuntu installation.
8. Boot the guest OS and verify that Ubuntu is functioning normally.
9. Record the allocated CPU, memory, disk, and virtualization settings as part of the observation.



Figure 4: Oracle VM VirtualBox Manager

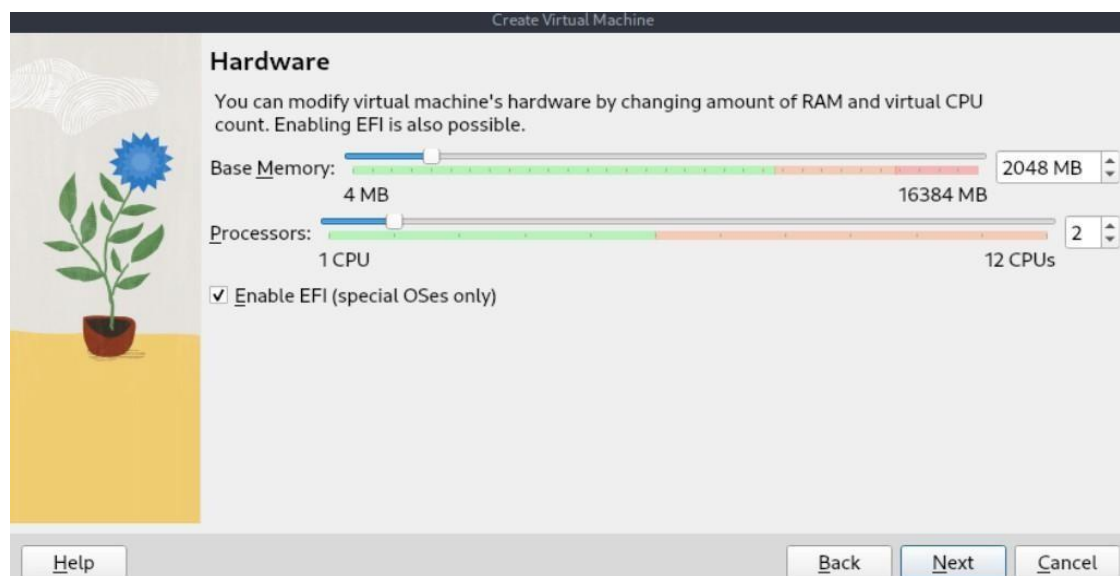


Figure 5: Allocation of virtual-machine resources

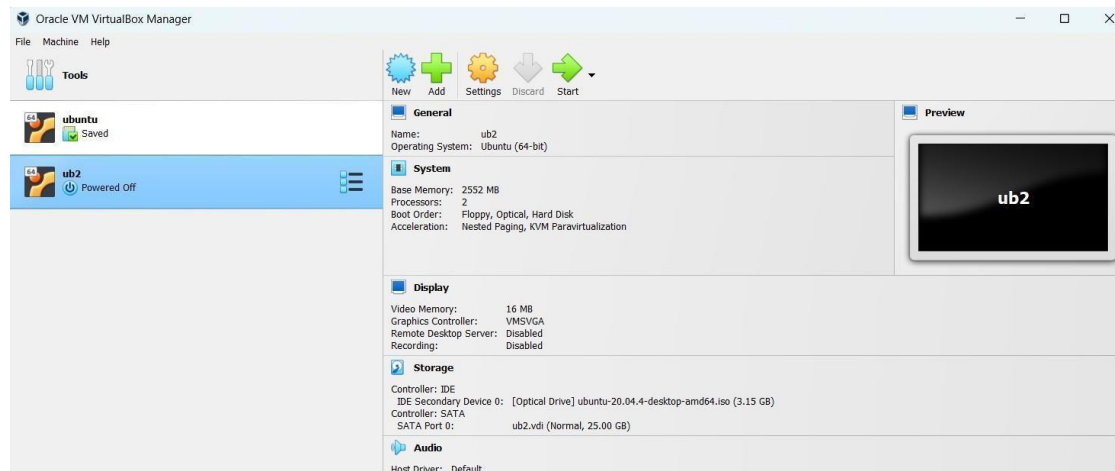


Figure 6: Virtual machine configured in VirtualBox

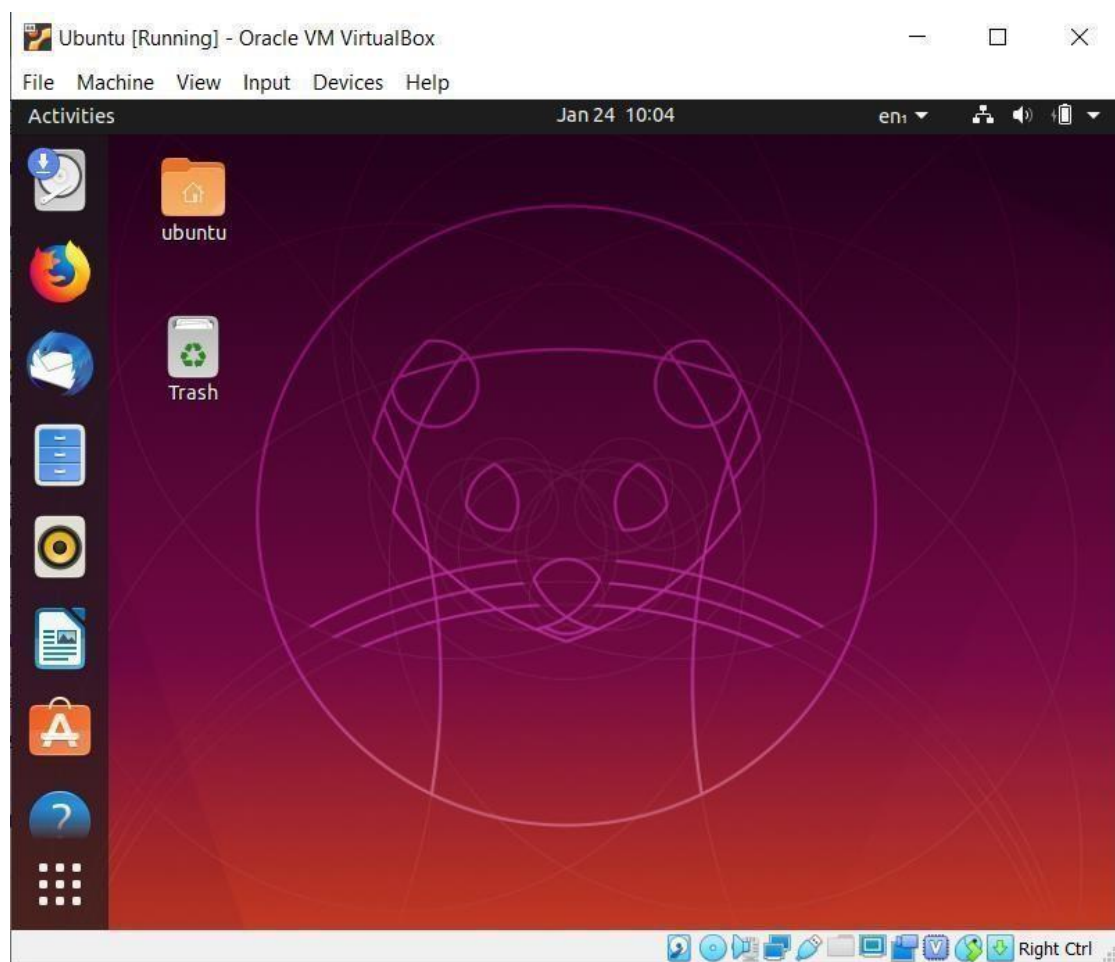


Figure 7: Ubuntu running as the guest operating system

Conclusion

The lab demonstrated how a virtual machine can be created and operated using Oracle VM VirtualBox. Ubuntu was installed as a guest OS, providing a practical foundation for the following experiment in which C and Java programs are executed inside the virtual machine.

Lab 3: Running C and Java Programs in a Type 2 Virtual Machine

Objectives

- To run a basic C program and a basic Java program inside a virtual machine.

Theory

A Type 1 hypervisor runs directly on the physical hardware, whereas a Type 2 or hosted hypervisor runs as software on top of a host operating system. The supplied report identifies Oracle VM VirtualBox as a Type 2 hypervisor and uses the Ubuntu VM created in Lab 2.

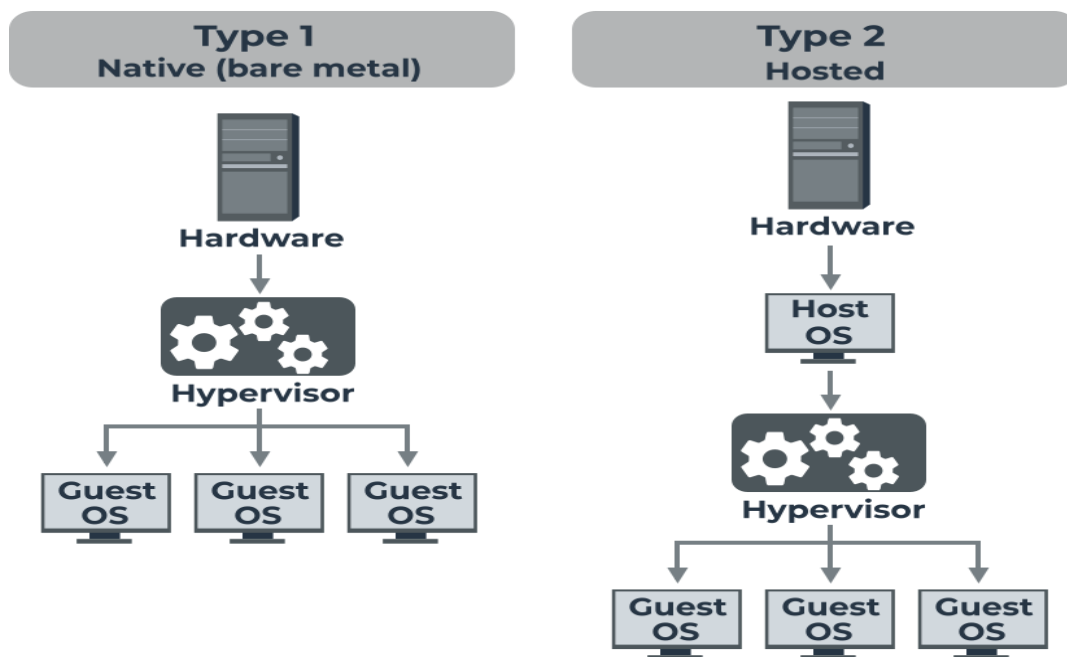


Figure 8: Type 1 and Type 2 hypervisors

Steps to Run a Java Program

- Install a Java Development Kit if required.
- Create a Java source file named hello.java.
- Compile the source file using javac.
- Execute the generated class using java.

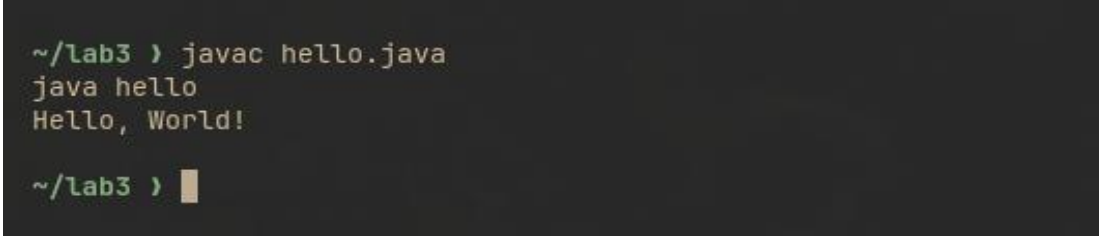
Steps to Run a C Program

- Install GCC if it is not already available.
- Create a source file named hello.c.
- Compile the program using gcc.
- Run the generated executable.

Source Code

```
// hello.java

class hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

A terminal window with a dark background. The prompt is ~/lab3 >. The user enters javac hello.java, then java hello, and the output is Hello, World!. The prompt returns to ~/lab3 >.

```
~/lab3 > javac hello.java
java hello
Hello, World!

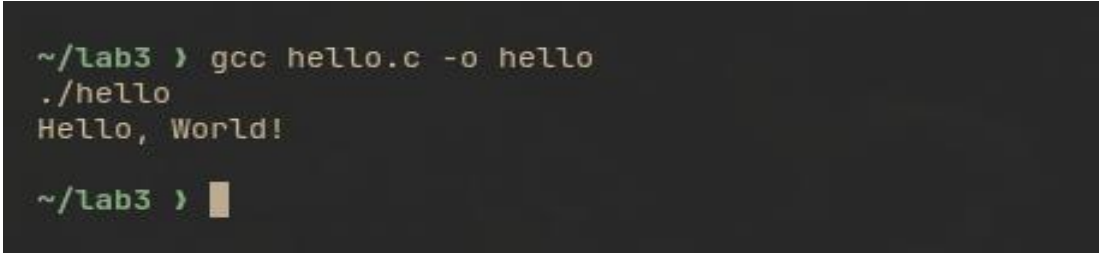
~/lab3 > █
```

Figure 9: Java program compilation and execution output

```
// hello.c

#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

A terminal window with a dark background. The prompt is ~/lab3 >. The user enters gcc hello.c -o hello, then ./hello, and the output is Hello, World!. The prompt returns to ~/lab3 >.

```
~/lab3 > gcc hello.c -o hello
./hello
Hello, World!

~/lab3 > █
```

Figure 10: C program compilation and execution output

Conclusion

The C and Java programs were compiled and executed inside the Ubuntu virtual machine. The experiment reinforced the concept of Type 2 virtualization and showed how a guest operating system can provide a practical programming environment.

Lab 4: Installation and Configuration of Hadoop

Objectives

- To understand the basic architecture of Apache Hadoop.
- To install and configure a single-node Hadoop environment.

Theory

Apache Hadoop is an open-source framework for distributed storage and processing of large datasets. The supplied report focuses on Hadoop 1.x architecture, where HDFS uses NameNode and DataNode components and MapReduce uses JobTracker and TaskTracker components.

Compatibility note: Hadoop 1.2.1 is an old Hadoop release. The original report combines that legacy release with a modern Ubuntu environment. Such a setup may require compatibility adjustments, especially for Java versions and older Hadoop scripts.

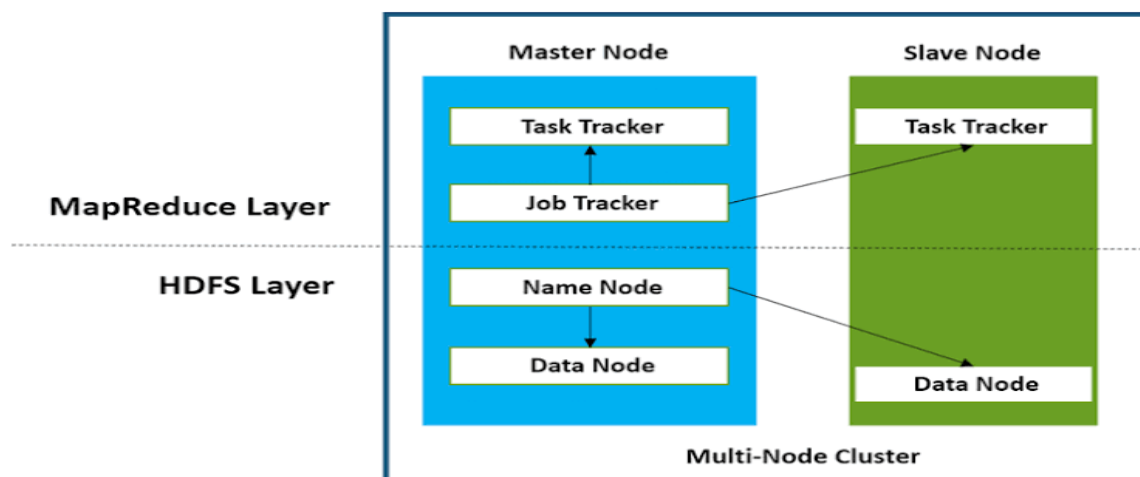


Figure 11: Hadoop architecture used in the lab

5. Installation and Configuration

5.1 Java Environment

The source report uses Java 8 and configures JAVA_HOME, JRE_HOME, and HADOOP_HOME. The following is a cleaned presentation of the same configuration approach.

```
sudo mkdir -p /usr/local/java
cd ~/Downloads
sudo cp jdk-8-linux-i586.tar.gz /usr/local/java
```

```

sudo cp jre-8-linux-i586.tar.gz /usr/local/java
cd /usr/local/java
sudo tar xvzf jdk-8-linux-i586.tar.gz
sudo tar xvzf jre-8-linux-i586.tar.gz
JAVA_HOME=/usr/local/java/jdk1.8.0
JRE_HOME=/usr/local/java/jre1.8.0
HADOOP_HOME=/home/hadoop/hadoop-1.2.1
PATH=$PATH:$HOME/bin:$JAVA_HOME/bin:$JRE_HOME/bin:$HADOOP_
HOME/bin
export JAVA_HOME
export JRE_HOME
export PATH
java -version

```

Hadoop Installation

- Create a Hadoop directory and extract the Hadoop distribution.
- Set JAVA_HOME in conf/hadoop-env.sh.
- Run Hadoop first in standalone mode to verify the basic installation.
- Create the required XML configuration files for pseudo-distributed operation.
- Configure passwordless SSH for localhost if required by the selected Hadoop release.
- Format the NameNode.
- Start the Hadoop services.
- Run jps and record the services displayed.

Core Configuration

```

<!-- conf/core-site.xml -->
<configuration>
  <property>
    <name>fs.default.name</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
<!-- conf/hdfs-site.xml -->
<configuration>
  <property>
    <name>dfs.replication</name>

```

```
        <value>1</value>
    </property>
</configuration>
<!-- conf/mapred-site.xml -->
<configuration>
    <property>
        <name>mapred.job.tracker</name>
        <value>localhost:9001</value>
    </property>
</configuration>
```

The replication factor is set to 1 because this is a single-node laboratory setup. In a real multi-node cluster, the replication strategy would normally be selected according to reliability and storage requirements.

Discussion

The single-node configuration is useful for learning because all major Hadoop components operate on one computer. It does not provide the same scalability or fault tolerance as a real distributed cluster. The experiment therefore demonstrates Hadoop's architecture and workflow rather than the performance of a production cluster.

Conclusion

Hadoop was studied and configured as a single-node environment, including Java setup, HDFS configuration, MapReduce configuration, and service verification. The lab provides the infrastructure required for the MapReduce word-count experiment in Lab 5.

Lab 5: Simple Word Count using Hadoop MapReduce

Objectives

- To understand the MapReduce processing model.
- To implement a simple word-count application.

Theory

MapReduce divides data processing into stages. The mapper processes input records and emits intermediate key-value pairs. The shuffle and sort phase groups values belonging to the same key, and the reducer combines those values to generate the final result. The supplied report uses the classic word-count example in which a mapper emits (word, 1) and a reducer sums the values for each word.

Stage	Input / Operation	Output
Map	Read each line and split it into words.	(word, 1)
Shuffle and Sort	Group identical keys.	(word, [1,1,1,...])
Reduce	Sum the values for each word.	(word, total)

Mapper Program

```
package org.mapreduce;
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;

public class WC_Mapper extends MapReduceBase
    implements Mapper<LongWritable, Text, Text, IntWritable> {

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
```

```

public void map(LongWritable key, Text value,
    OutputCollector<Text, IntWritable> output,
    Reporter reporter) throws IOException {

    String line = value.toString();
    StringTokenizer tokenizer = new StringTokenizer(line);

    while (tokenizer.hasMoreTokens()) {
        word.set(tokenizer.nextToken());
        output.collect(word, one);
    }
}
}

```

Reducer Program

```

package org.mapreduce;
import java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;

public class WC_Reducer extends MapReduceBase
    implements Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterator<IntWritable> values,
        OutputCollector<Text, IntWritable> output,
        Reporter reporter) throws IOException {

        int sum = 0;

        while (values.hasNext()) {

```

```

        sum += values.next().get();
    }

    output.collect(key, new IntWritable(sum));
}
}

```

Runner Program

```

package org.mapreduce;
import java.io.IOException;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.mapred.TextInputFormat;
import org.apache.hadoop.mapred.TextOutputFormat;

public class WC_Runner {

    public static void main(String[] args) throws IOException {
        JobConf conf = new JobConf(WC_Runner.class);
        conf.setJobName("WordCount");

        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);

        conf.setMapperClass(WC_Mapper.class);
        conf.setCombinerClass(WC_Reducer.class);
        conf.setReducerClass(WC_Reducer.class);

        conf.setInputFormat(TextInputFormat.class);
        conf.setOutputFormat(TextOutputFormat.class);
    }
}

```

```

FileInputFormat.setInputPaths(conf, new Path(args[0]));
FileOutputFormat.setOutputPath(conf, new Path(args[1]));

JobClient.runJob(conf);
}
}

```

Run the Job

```

hadoop jar target/mapreduce-1.0-SNAPSHOT.jar org.mapreduce.WC_Runner
/input/random.txt /output

```

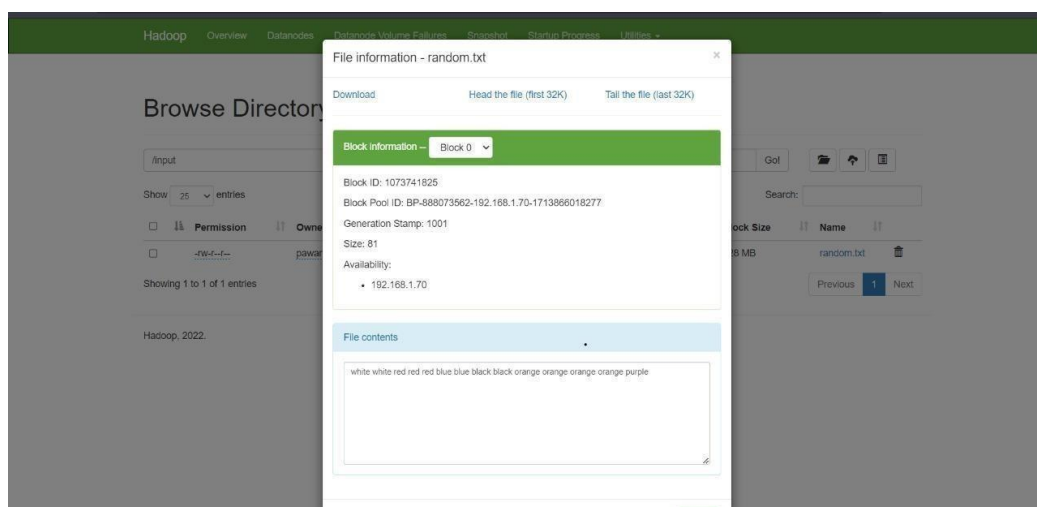


Figure 12: HDFS input file / random.txt

```

PS C:\Users\pawan\IdeaProjects\mapreduce> hadoop jar target/mapreduce-1.0-SNAPSHOT.jar org.pawan.WC_Runner /input/random.txt /output
2024-04-23 15:42:49,291 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
2024-04-23 15:42:49,462 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
2024-04-23 15:42:49,899 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2024-04-23 15:42:49,914 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/pawan/.staging/job_1713866077325_0001
2024-04-23 15:42:50,107 INFO mapred.FileInputFormat: Total input files to process : 1
2024-04-23 15:42:50,607 INFO mapreduce.JobSubmitter: number of splits:2
2024-04-23 15:42:50,776 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1713866077325_0001

```

Figure 13: MapReduce job execution output

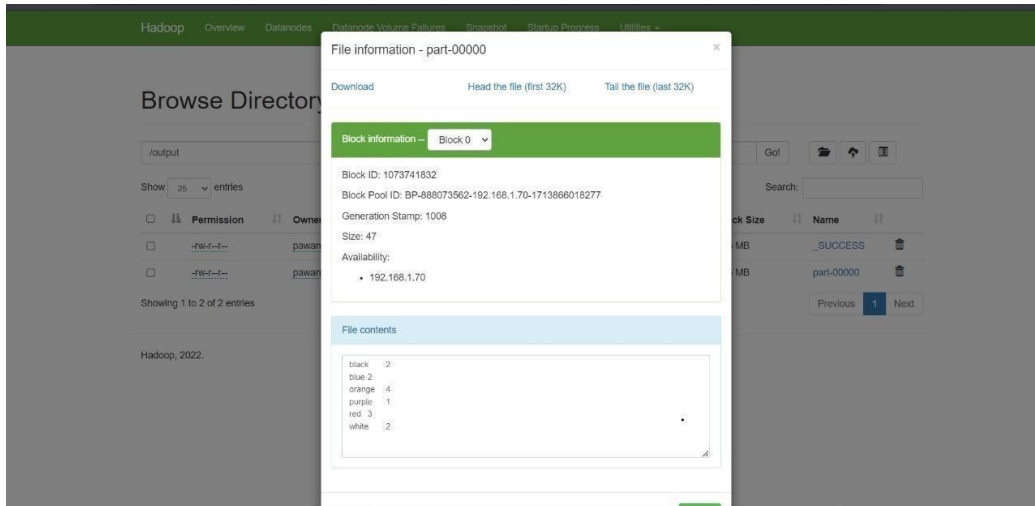


Figure 14: HDFS word-count output

Output Interpretation

For the sample input shown in the source report, the expected counts are: black = 2, blue = 2, orange = 3, purple = 1, red = 3, and white = 2.

Word	Count
black	2
blue	2
orange	3
purple	1
red	3
white	2

Conclusion

A simple word-count MapReduce application was studied and executed conceptually through the Hadoop environment. The experiment demonstrates the complete MapReduce flow from input data to mapper output, shuffle and sort, reduction, and final HDFS output.

Lab 6: Case Study – Google Workspace

Objectives

- To understand Google Workspace as a SaaS platform.

Theory

Google Workspace is a collection of cloud-based productivity and collaboration services. The supplied report identifies Gmail, Google Docs, Google Drive, Google Sheets, Google Slides, Google Meet, Google Calendar, Google Chat, and Google Forms as major services. It uses Google Workspace as the practical example of SaaS.



Figure 15: Major Google Workspace applications

Development / History

The source report describes the service as beginning in 2006 as Google Apps for Your Domain, becoming G Suite in 2016, and being renamed Google Workspace in 2020. These historical points are included as part of the supplied case study.

Major Applications

Application	Main use
Gmail	Email communication
Google Docs	Collaborative document creation and editing
Google Sheets	Spreadsheet-based data management and analysis
Google Slides	Collaborative presentation creation
Google Drive	Cloud file storage and sharing
Google Meet	Online meetings and video communication
Google Calendar	Scheduling and event management
Google Chat	Team and group messaging
Google Forms	Surveys, quizzes and data collection

SaaS Responsibility Model

Generally managed by provider	Generally managed by customer/administrator
Servers and underlying infrastructure	User accounts and access settings
Application software and updates	Organization-level sharing policies
Data-center infrastructure	User-created content
Service-side security mechanisms	Which users can access which organizational resources

The exact responsibility boundary depends on the service, organization settings, subscription, and security configuration. The table is therefore intended as a conceptual SaaS responsibility model rather than a complete legal or technical responsibility matrix.

Limitations

- Dependence on network connectivity for many workflows
- Data and service dependence on the provider
- Subscription costs can increase with organizational user count
- Migration can become more complex when an organization depends heavily on provider-specific services

Security Considerations

- Multi-factor or two-step verification
- Encryption in transit and at rest
- Administrative sharing and access controls
- Retention and e-discovery features where available in the applicable plan

Application in Education

The source report presents a college use case in which teachers and students can use cloud tools for assignments, notes, online meetings, collaborative reports, and quizzes. Such a model reduces the need for an institution to maintain every productivity service.

Conclusion

Google Workspace provides a practical example of SaaS because users interact primarily with finished cloud applications while the provider manages much of the underlying software and infrastructure. The case study also demonstrates how SaaS can support collaboration and educational activities.

Lab 7: Case Study – Google App Engine

Objectives

- To understand Google App Engine as a PaaS platform.
- To understand the relationship between application code and managed cloud infrastructure.

Theory

Google App Engine (GAE) is presented in the source report as a PaaS service. In the PaaS model, developers concentrate on application code and deployment while the cloud provider manages much of the underlying server infrastructure, operating environment, scaling, and platform maintenance.

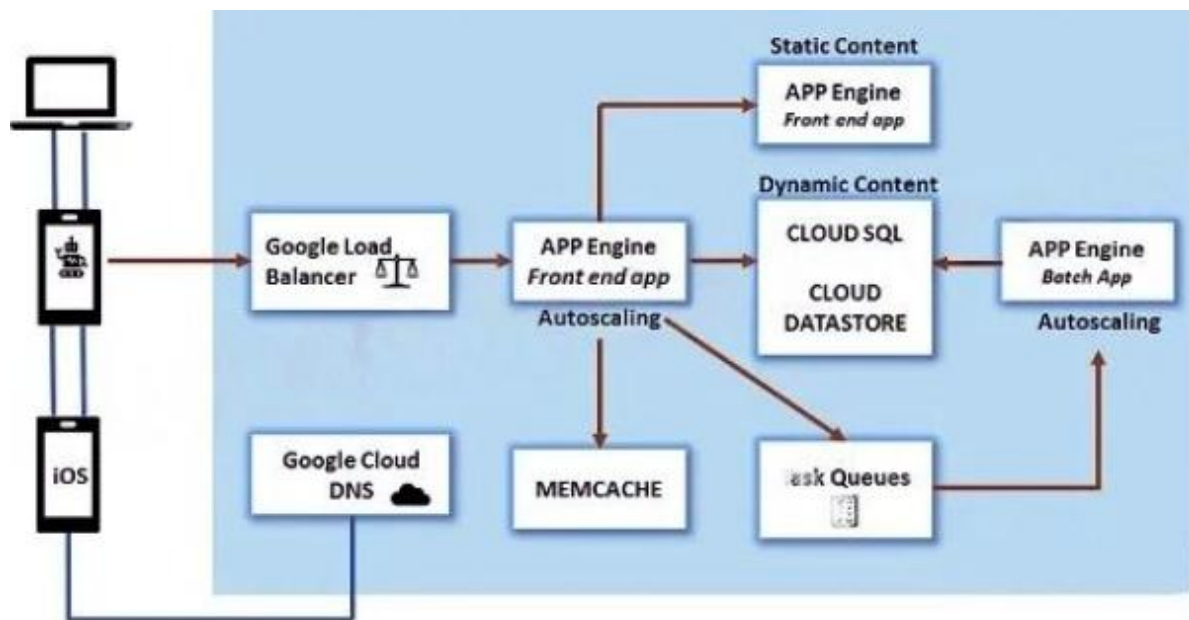


Figure 16: Conceptual architecture of a Google App Engine application

Application Flow

- A client sends a request to the application.
- The request is handled by the cloud platform's routing/load-balancing layer.
- An appropriate application instance processes the request.
- The application communicates with required data or storage services.
- The platform can create, stop, or route traffic among application instances according to the selected environment and workload.

Environments

Aspect	Standard environment	Flexible environment
Execution model	Managed sandbox environment	Container-based environment on managed infrastructure
Language/customization	Supported runtimes	Greater flexibility through custom containers
Scaling behavior	Can scale down when there is no traffic in supported configurations	Designed for applications requiring more flexibility
Startup characteristics	Generally faster startup	Generally more infrastructure-oriented startup behavior

The original report distinguishes Standard and Flexible environments and notes that Standard uses a managed sandbox while Flexible uses Docker containers on Compute Engine virtual machines.

Main Components

Component	Purpose
Application	Top-level App Engine application associated with a Google Cloud project.
Service	Logical component of an application, such as a web service or API.
Version	A deployed version of application code.
Instance	A running copy of a deployed version.

Deployment Procedure

- Create or select a Google Cloud project.
- Install and authenticate the Google Cloud CLI.
- Prepare the application source code.
- Create an app.yaml file describing the runtime configuration.
- Deploy the application using the appropriate gcloud command.
- Open the deployed application and verify the result.
- Record the deployment output and URL as evidence.

Benefits

- Managed infrastructure
- Automatic scaling capabilities
- Reduced server-management effort
- Version-based deployment
- Integration with other cloud services

Limitations

- Less infrastructure-level control than IaaS
- Potential dependence on provider-specific services
- Platform and environment constraints
- Application migration can require changes when provider-specific features are heavily used

Conclusion

Google App Engine was studied as a PaaS platform, including its architecture, environments, components, deployment process, benefits, and limitations. The case study demonstrates how cloud platforms can abstract infrastructure management from application development.

Lab 8: Case Study – Disaster Recovery Center (DRC), Hetauda

Objectives

- To see how disaster recovery is done in a real cloud environment.

Theory:

As part of the Cloud Computing course, we visited the Disaster Recovery Center (DRC) in Hetauda. It is an important part of Nepal's digital infrastructure. The visit showed us how cloud systems, virtualization and data replication keep important services running during failures or disasters.

About the Disaster Recovery Center (DRC)

The DRC in Hetauda is a backup site. It keeps a copy of important data and services from the Government Integrated Data Center (GIDC) in Kathmandu. If the main site fails, services move to DRC. It uses cloud and virtualization technology to bring services back quickly, so government services can continue.



Fig. 18: The Disaster Recovery Center building in Hetauda

Key Technical Areas

Virtualization

- The report describes hypervisors such as VMware and KVM as technologies used to create virtual machines.
- Virtualization allows physical computing resources to be shared among multiple virtual machines.

Data Replication and Backup

- The report describes replication of data between the primary and backup locations.
- It describes full and incremental backup approaches.
- It identifies RAID as a storage-reliability mechanism.

Network Infrastructure

- The source report describes a dedicated fiber connection between Kathmandu and Hetauda.
- It describes redundant network paths to reduce the effect of individual failures.
- It identifies load balancing and failover as mechanisms for maintaining service availability.

Cloud Security

- Firewalls and intrusion-detection mechanisms are described as part of the security environment.
- Role-based access control is identified as an access-management mechanism.
- Encryption is described for data in transit and at rest.
- Security checks and disaster-recovery drills are identified as important operational activities.

Cloud Deployment Perspective

The source report characterizes the environment mainly as private cloud because it is associated with government-managed infrastructure, while also describing characteristics it interprets as hybrid because services and infrastructure are distributed across locations.

Disaster Recovery Concepts Learned

Concept	Purpose
Backup	Creates recoverable copies of data.
Replication	Maintains copies of data or workloads at another location.
Redundancy	Provides alternate components or paths when one component fails.
Failover	Moves service operation to an alternate resource/site after a failure.
Recovery testing	Checks whether systems and procedures can actually restore services.

Conclusion

The DRC case study demonstrated how virtualization, backup, replication, network redundancy, security, and disaster-recovery practices can work together to support service continuity. The source report concludes that these technologies can help maintain digital services during unexpected disruptions.